

λ -calculus

Programação Funcional



λ -calculus

- Alonzo Church
 - Professor em Princeton e UCLA (1929-1990)
 - Inventou o λ -calculus (Lambda Cálculo)
 - Definiu a noção de função computável
- A menor linguagem de programação universal
- Linguagens de programação funcionais, como Lisp, Miranda, ML, Haskell são baseadas no λ -calculus
- Utilizada como linguagem intermediária no processo de compilação de linguagens funcionais



Sintaxe

- O lambda cálculo possui apenas quatro construções:
 - Constantes, variáveis, aplicações e abstrações
- Uma expressão lambda pode ser
$$\begin{aligned} &< \lambda - expr > :: < constante > \\ &| < variavel > \\ &| (< \lambda - expr >) (< \lambda - expr >) \\ &| (\lambda < variavel > . < \lambda - expr >) \end{aligned}$$
- Os parênteses podem ser utilizados nas expressões para prevenir ambiguidades



Exemplos

1. z – variável
2. 8 – constante
3. zy – combinação ou aplicação
4. $\lambda a . ab$ – λ -abstração ou função
5. $(\lambda y . y)(\lambda a . ab)$ – combinação ou aplicação
6. $(a(\lambda z . y))a$ – combinação ou aplicação
7. $(\lambda a . \lambda b . \lambda c . ac)(\lambda k . k)(\lambda x . x)$ – combinação ou aplicação



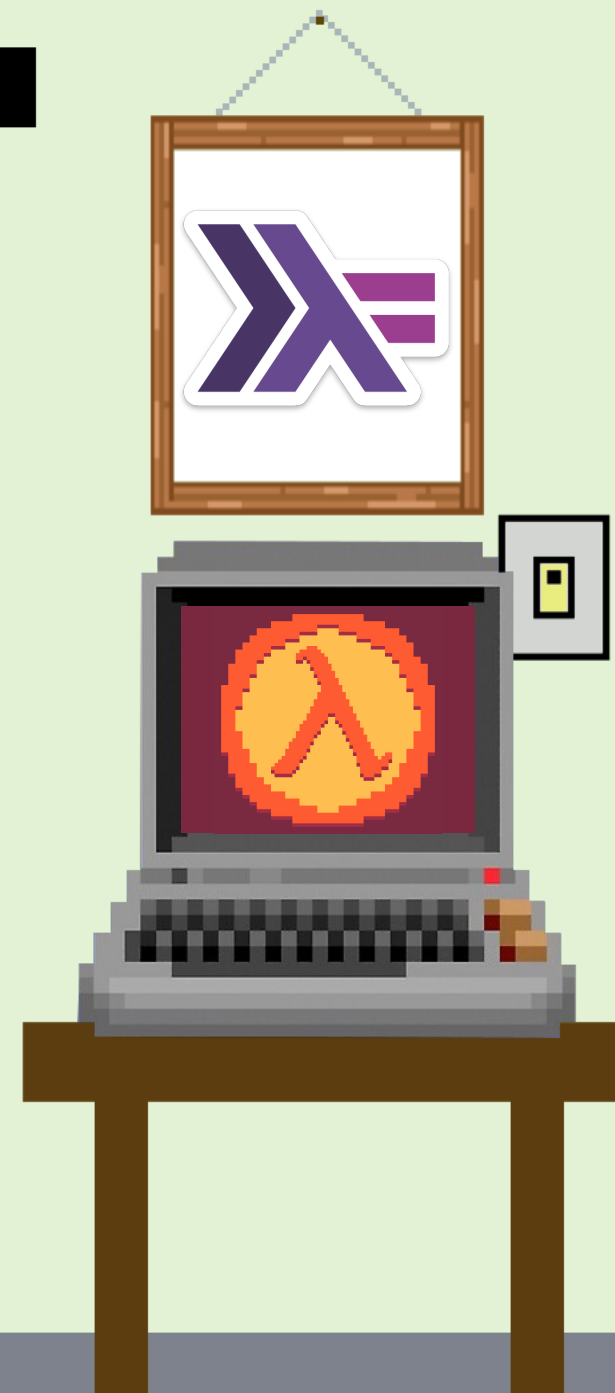
Sintaxe

- A expressão λ se estende para a direita
 - $\lambda f.xy \rightarrow \lambda f.(xy)$
- A aplicação é associativa à esquerda
 - $xyz \rightarrow (xy)z$
- Múltiplos lambdas podem ser omitidos
 - $\lambda f.\lambda g.x \rightarrow \lambda fg.x$



Funções Pré-definidas

- Originalmente o λ -cálculo não tem funções embutidas
- Na prática, algumas funções foram embutidas
 - Operações aritméticas: +, -, *, /
 - Constantes: 0, 1, 2, 3, ... 9, True, False
 - Operadores lógicos: AND, OR, NOT
 - Função condicional
 - IF True E1 E2 $\rightarrow$ E1
 - IF False E1 E2 $\rightarrow$ E2



λ -abstrações

- Funções não embutidas na linguagem
- Construídas anonimamente
- Ex: $\lambda x. + x 1$
 - λ - indica que se trata de uma função
 - x - sobre a variável x
 - $.$ - que
 - $(+ x 1)$ - adiciona x ao número 1



Aplicação de Função e Currificação

- No λ -cálculus a aplicação de uma função f a um parâmetro x é denotada por justaposição, ou seja: $f x$
- Funções com mais de um argumento são simplificadas
 - Ex: $+ 3 4$ é interpretada como $(+ 3) 4$
- Currificação
 - O resultado de uma função pode ser outra função
 - Descoberta por Schönfinkel e utilizada por Curry
 - Uma função com n argumentos pode ser vista como n funções de 1 (um) argumento

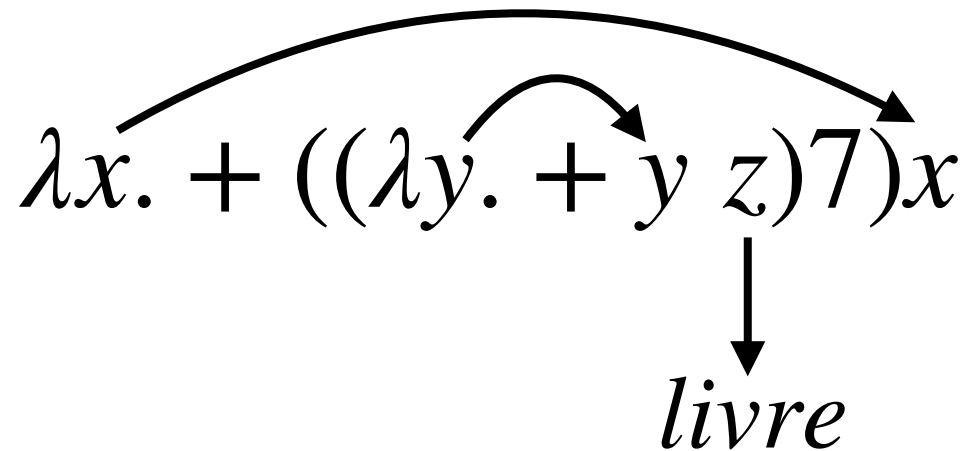


Variáveis Livres e Ligadas

- Seja a λ -expressão $(\lambda x. + x y)4$
- Aplicação de uma função sobre a variável x , com corpo $(+ x y)$, a uma outra λ -expressão (constante 4)
 - A variável x pode ser pensada como um local onde o argumento 4 deve ser colocado
 - O mesmo não pode ser feito com a variável y
 - As variáveis x e y tem status diferente (a variável x está ligada, enquanto que y é livre)
- Variáveis ligadas são aquelas que possuem vínculo com alguma λ -abstração
 - Caso não possuam nenhum vínculo são livres



Exemplo



- Uma variável pode ocorrer livre em um ponto da λ -abstração e ligado em outro
 - $+x((\lambda x. + x 1)4)$



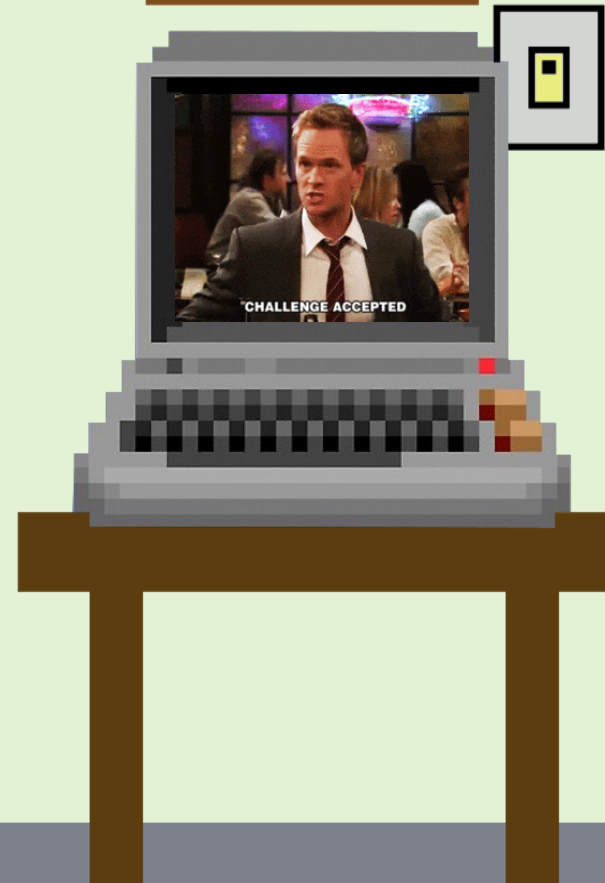
Combinadores

- Uma λ -expressão que não apresenta variáveis livres é chamada fechada ou combinador
- Alguns combinadores desempenham um papel especial no estudo do λ -calculus
- Exemplos
 - $I = \lambda x . x$ - Identidade
 - $K = \lambda xy . x$ - Projeção
 - $S = \lambda xyz . xz(yz)$ - Composição
 - $\Delta = \lambda x . xx$ - Duplicação
 - $Y = \lambda f . (\lambda y . f(yy))(\lambda y . f(yy))$ - Usado em recursão
 - $\Theta = \lambda ab . b(aab)$ - Também usado em recursão



Exercícios

- Identifique as λ -expressões:
 - a
 - 9
 - $((\lambda b . b)(\lambda a . ab))$
 - $\lambda z . xzzx$
 - $(\lambda x .)a$
 - $((\lambda xy . y)(\lambda y . yyy))((\lambda i . i)(\lambda a . b))$



Exercícios

- Identifique as λ -expressões:
 - a - variável
 - 9 - constante
 - $((\lambda b . b)(\lambda a . ab))$ - inválida, parênteses sobrando
 - $\lambda z . xzzx$ - λ -abstração ou função
 - $(\lambda x .)a$ - inválida, sem corpo
 - $((\lambda xy . y)(\lambda y . yyy))((\lambda i . i)(\lambda a . b))$ - combinação



Exercícios

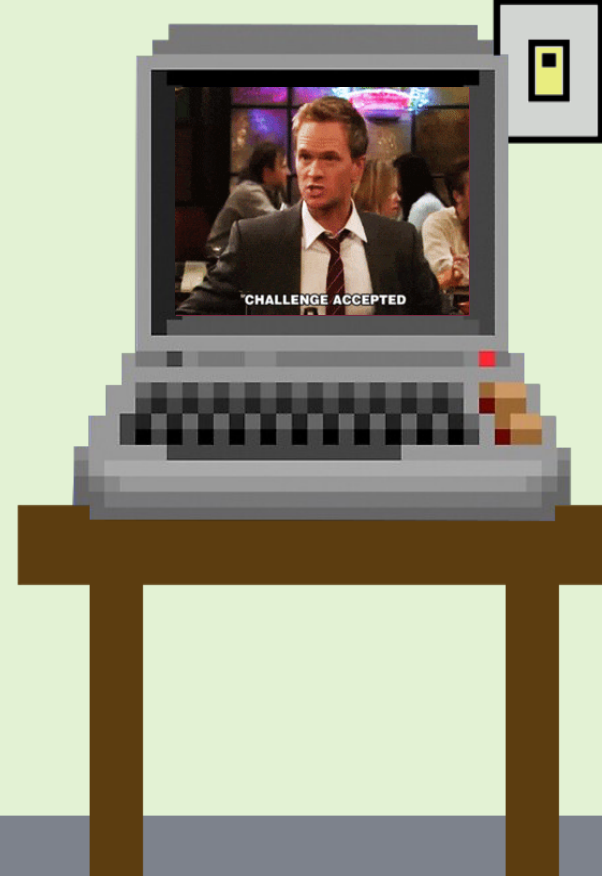
- Identifique as ocorrências livres e ligadas das variáveis nas expressões abaixo:

- $\lambda x . xx$

- $(\lambda xy . x)x$

- $(\lambda xy . xx)xa$

- $(x(\lambda x . y))x$



Exercícios

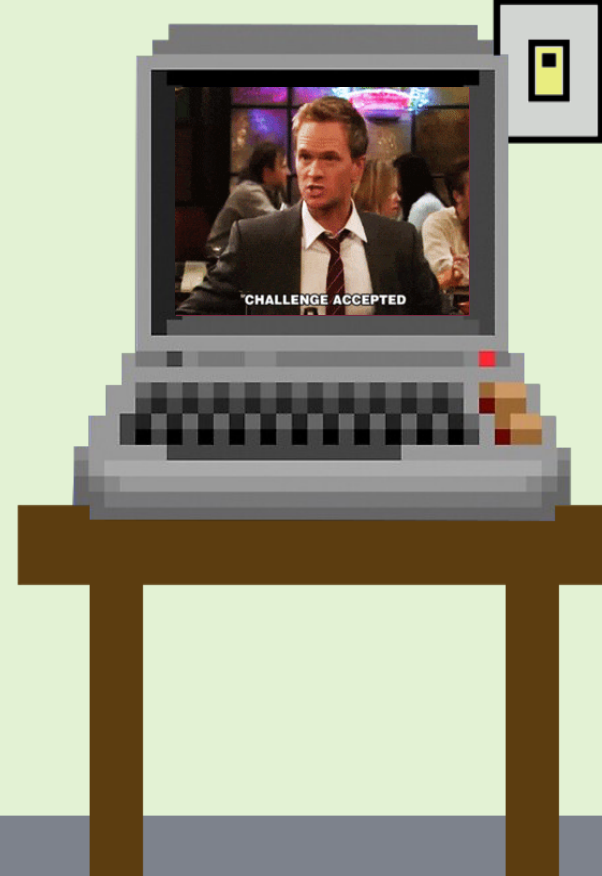
- Identifique as ocorrências livres e ligadas das variáveis nas expressões abaixo:

- $\lambda x. \underline{xx}$


- $(\lambda xy. x)x$


- $(\lambda xy. \underline{xx})xa$


- $(x(\lambda x. y))x$



Conversões de λ -expressões

- O processo de avaliação do λ -cálculo transforma uma λ -expressão em outra geralmente mais simples
 - Repetição até que a λ -expressão não possa mais ser transformada ou entre em loop infinito
- Técnicas de transformação:
 - α -conversão, β -conversão e η -conversão
 - Semelhante a técnicas algébricas utilizadas em equações matemáticas



α -conversão

- Considere as duas λ -abstrações
 - $(\lambda x. + x 1)$ e $(\lambda y. + y 1)$
 - As duas são equivalentes e uma α -conversão muda o nome dos parâmetros formais
- Logo
 - $(\lambda x. + x 1) \leftrightarrow (\lambda y. + y 1)$
- A α -conversão determina uma relação de equivalência entre duas λ -expressões
- A duas λ -expressões representam a mesma entidade



α -conversão

■ Exemplos

- $(\lambda x. * x 1) \xrightarrow{\alpha} (\lambda y. * y 1)$

- $\lambda x. x \xrightarrow{\alpha} \lambda y. y \xrightarrow{\alpha} \lambda z. z$

- $\lambda x. axa \xrightarrow{\alpha} \lambda b. aba$

■ Pode-se deduzir as seguintes relações (Equivalência)

- $\theta \xrightarrow{\alpha} \theta$ (Reflexão)

- $\theta \xrightarrow{\alpha} \delta \Leftrightarrow \delta \xrightarrow{\alpha} \theta$ (Simetria)

- $\theta \xrightarrow{\alpha} \delta$ e $\delta \xrightarrow{\alpha} \gamma \Rightarrow \theta \xrightarrow{\alpha} \gamma$ (Transitividade)



β -conversão

- A aplicação de um argumento à uma λ -abstração resulta em uma instância do corpo desta abstração
- Neste caso, as ocorrências livres do parâmetro formal nesse corpo são trocadas por cópias do argumento
 - Ex1: $(\lambda x. + x\ 1)4 \rightarrow +\ 4\ 1 \rightarrow 5$
 - Ex2: $(\lambda x. + x\ 1)7 \rightarrow +\ 7\ 1 \rightarrow 8$
- Observação
 - Apenas as respectivas ocorrências da variável ligada devem ser substituídas
- Funções também podem ser usadas como argumento
 - Ex: $(\lambda f.f\ 3)(\lambda x. + x\ 1) \xrightarrow{\beta} (\lambda x. + x\ 1)3 \xrightarrow{\beta} +\ 3\ 1 = 4$



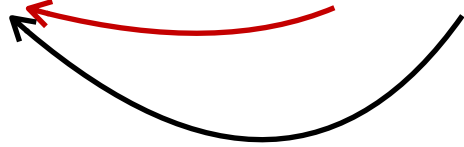
Captura

- Cuidado na escolha dos nomes dos parâmetros, uma vez que mais de um deles pode ter o mesmo nome
 - Recomenda-se realizar α -conversão
- Exemplo
$$(\lambda x. (\lambda x. + (-x\ 1))x\ 3)9$$



Captura

- Cuidado na escolha dos nomes dos parâmetros, uma vez que mais de um deles pode ter o mesmo nome
 - Recomenda-se realizar α -conversão
- Exemplo
 $(\lambda x. (\lambda x. + (-x\ 1))x\ 3)9$ [Erro!]



Captura

- Cuidado na escolha dos nomes dos parâmetros, uma vez que mais de um deles pode ter o mesmo nome
 - Recomenda-se realizar α -conversão

- Exemplo

$(\lambda x. (\lambda x. + (-x\ 1))x\ 3)9 \xrightarrow{\beta} (\lambda x. + (-9\ 1))9\ 3$ [Erro!]



Captura

- Cuidado na escolha dos nomes dos parâmetros, uma vez que mais de um deles pode ter o mesmo nome
 - Recomenda-se realizar α -conversão

- Exemplo

$$(\lambda x. (\lambda x. + (-x\ 1))x\ 3)9 \xrightarrow{\alpha} (\lambda x. (\lambda y. + (-y\ 1))x\ 3)9$$

$$(\lambda x. (\lambda y. + (-y\ 1))x\ 3)9 \xrightarrow{\beta} (\lambda y. + (-y\ 1))9\ 3$$

$$(\lambda y. + (-y\ 1))9\ 3 \xrightarrow{\beta} + (-9\ 1)3$$

$$+8\ 3 \rightarrow 11$$



Ordem de Redução

- Redex é uma λ -expressão capaz de ser reduzida
 - Pode sofrer uma β -conversão
- Se uma expressão não contiver qualquer redex a sua avaliação está completa (forma normal)
- A ordem de reduções determina diferenças
 - $\Delta = (\lambda x.xx)$ - Combinador de duplicação
 - $(\lambda y.7)(\Delta\Delta)$
 - $(\lambda x.xx)(\lambda x.xx) \xrightarrow{\beta} (\lambda x.xx)(\lambda x.xx) \xrightarrow{\beta} \dots$
 - 7
- Leftmost-outermost é a ordem normal de redução
 - Define qual redex deve ser escolhido primeiro



η -conversão

- Considere as duas expressões: $(\lambda x. + 1 x)$ e $(+1)$
 - As duas se comportam igualmente quando aplicadas ao mesmo argumento
 - Elas também podem ser convertidas uma na outra
 - $(\lambda x. + 1 x) \rightarrow (+1)$
- Reflete o princípio da extensionalidade
 - Duas funções são iguais se elas apresentam os mesmos resultados para as mesmas entradas
- Logo
 - $(\lambda x. F x) \leftrightarrow F$



Resumo

- α -conversão
 - Permite trocar o nome de um parâmetro
- β -conversão
 - Permite aplicar uma λ -abstração a um argumento
- η -conversão
 - Permite eliminar λ -abstrações redundantes



Aplicando o λ -calculus

- $True = \lambda xy . x$
- $False = \lambda xy . y$
- $NOT = \lambda b . b \ False \ True$
- Aplicação $NOT \ True \rightarrow False$
 - $NOT \ True$
 - $(\lambda b . b \ False \ True)True$
 - $(\lambda b . b \ (\lambda xy . y) \ (\lambda xy . x))True$
 - $(True \ (\lambda xy . y) \ (\lambda xy . x))$
 - $(\lambda xy . x)(\lambda xy . y)(\lambda xy . x)$
 - $\lambda xy . y$
 - $False$



Aplicando o λ -calculus

- $True = \lambda xy . x$
- $False = \lambda xy . y$
- $NOT = \lambda b . b \ False \ True$
- Aplicação $NOT \ False \rightarrow \ True$
 - $NOT \ False$
 - $(\lambda b . b \ False \ True)False$
 - $(\lambda b . b \ (\lambda xy . y) \ (\lambda xy . x))False$
 - $(False \ (\lambda xy . y) \ (\lambda xy . x))$
 - $(\lambda xy . y)(\lambda xy . y)(\lambda xy . x)$
 - $\lambda xy . x$
 - $True$



Exercício

- Coloque as λ -expressões abaixo em sua forma normal
 - $(\lambda xy. xxy)(\lambda y. y)7$
 - $(\lambda xy. xy)(\lambda z. zzz)w$
 - $(\lambda xyz. yzx)(5)(\lambda w. w)(+2)$

